Terminal Colour-Output Policy — Design Notes

Codex

June 24, 2026

Contents

1 Purpose

This document is the design rationale for how `clautolisp` and `alfe` decide whether to emit ANSI colour for AutoLISP value output, what colour to pick, and — most importantly — **how they probe the controlling terminal for that decision** without hanging or producing garbage.

It is the prose companion to:

- `clautolisp/autolisp-runtime/source/terminal-color.lisp` — the policy resolver (`RESOLVE-COLOR-POLICY`), the ANSI SGR helpers (`ANSI-COLORIZE` / `WRITE-ANSI-COLORIZED`), the cascade of env-var probes, and the OSC 11 helper.
- `clautolisp/autolisp-runtime/source/model.lisp` — the `AUTOLISP-SYMBOL PRINT-OBJECT` method that consults `*COLOR-OUTPUT*`.
- `clautolisp/documentation/clautolisp-specifications.org` — the user-facing spec entry for the same machinery.
- `clautolisp/documentation/clautolisp-user-manual.org` — the worked-example menu of env vars.

- `issues/closed/debug-messages-autolisp-symbols.issue` — the bug report that drove the original feature.

The feature is small in code volume but accumulated a few non-obvious decisions worth recording, because the **next** engineer working on it will be tempted (as I was) to "just turn the OSC 11 probe on by default" and immediately regress one of the supported platforms.

2 The original requirement

The bug that prompted the work was structural noise in error messages: an unbound-value error mentioning the AutoLISP symbol `T` was rendered as the full Common Lisp defstruct dump

```
#S(CLAUTOLISP.AUTOLISP-RUNTIME.INTERNAL:AUTOLISP-SYMBOL
                    :NAME "T" :ORIGINAL-NAME "T" :PLIST NIL)
```

rather than the AutoLISP surface name `T`. The plain-text fix was `PRINT-OBJECT` methods on every runtime value struct (closed in commit 385b74b). The colour portion — yellow on dark / blue on light terminal backgrounds, gated by `NO_COLOR` + a new `--no-color` flag — was deferred to a separate ticket because the **detection** of dark vs light is the hard part.

3 Constraints

1. **Never break a terminal session.** The colour feature is a UX nicety; any failure mode that leaves the user's tty in raw mode, garbles their prompt, or hangs the CLI is strictly worse than no colour at all.

2. **Compose with non-tty contexts.** Output captured to a pipe, a file, a test harness, or a backend's stdout-capture stream must never carry escape sequences. The colour policy must check `isatty` before doing anything.

3. **Honour `$NO_COLOR` (`https://no-color.org`).** The convention is universal: presence of the var disables colour, value content is ignored.

4. **Honour a per-invocation override.** `--no-color` exists for users who want suppression without exporting an env var.

5. **Portable across SBCL and CCL.** The implementation must compile and run on both — no reaching into a vendor-specific contrib that the other lacks.

6. **Forward-portable to MS-Windows.** clautolisp / alfe target the Windows-via-msys2 path today and bare Windows Terminal tomorrow. The implementation must not assume POSIX /dev/tty + tcsetattr semantics universally.

4 Architecture overview

`*COLOR-OUTPUT*` is a single special variable in `clautolisp.autolisp-runtime`. Its value is the entire policy:

- `NIL` — colour disabled. `PRINT-OBJECT` on `AUTOLISP-SYMBOL` writes the bare name.
- A keyword (`:YELLOW`, `:BLUE`, …) — foreground only. Backwards-compatible shape from the original implementation.
- A list `(FOREGROUND BACKGROUND)` — both SGR axes. Either element may be `NIL` to leave that axis bare.

The CLI computes the policy **once** at startup via `RESOLVE-COLOR-POLICY` and binds the variable for the duration of the run. Library code outside the CLI never flips it: the printer contract is observed by every error message, trace line, and REPL display, so an accidental binding would leak through the whole pipeline.

5 The five-step resolution order

In `RESOLVE-COLOR-POLICY` (terminal-color.lisp), in order:

1. `--no-color` (CLI flag) → `NIL`.
2. `$NO_COLOR` set to any non-empty value → `NIL`.
3. `STREAM` (defaults to `*standard-output*`) is not a tty → `NIL`. The isatty check uses `sb-alien` on SBCL and `ccl::isatty` on CCL, with `INTERACTIVE-STREAM-P` as a final fallback. Synonym streams (which is what `*standard-output*` usually is) are unwrapped first; without that, `sb-sys:fd-stream-fd` would not see the underlying fd.

4. `$CLAUTOLISP_SYMBOL_FOREGROUND` and/or `$CLAUTOLISP_SYMBOL_BACKGROUND` set → use them. This is the explicit per-axis override; it wins over the luminance probe so a user who wants a specific look gets it deterministically. Either or both env vars may be set; an unspecified axis stays bare in the SGR sequence.

5. Luminance probe (only when `$CLAUTOLISP_PROBE_OSC11` is set — see next section) → `:BLUE` for light backgrounds, `:YELLOW` otherwise. When the cascade returns nothing, fall back to `:YELLOW` (the dark default).

The order matters: explicit user overrides come before any probing, and probing is the **last** rung because the OSC 11 helper is the most expensive and the most fragile.

6 Why the OSC 11 probe is opt-in

6.1 The probe itself

OSC 11 (Operating System Command 11, also written `ESC ]11; ? ESC \`) is a terminal control sequence defined by xterm and adopted by most modern terminal emulators (iTerm2, GNOME Terminal, WezTerm, Alacritty, Kitty, recent Windows Terminal). The query asks the terminal "what is your background colour?" and the terminal responds inline with `ESC ]11;rgb:RRRR/GGGG/BBBB ESC \` (or BEL-terminated). Compute the Rec-601 luminance, threshold at 0.5, and you have a dark/light verdict.

The mechanics, however, are subtle:

- The query must be written to `/dev/tty`, **not** to the process's stdout (which may be redirected to a pipe).
- The response is read from `/dev/tty`, again bypassing stdin redirection.
- The terminal must be in non-canonical mode (`stty -icanon`) so the response bytes don't sit in the kernel line-buffer waiting for newline.
- Echo must be off (`-echo`) so the terminal doesn't print the response.
- There must be a **read timeout** — terminals that don't support OSC 11 simply don't reply, and we cannot block forever waiting.
- Inside tmux or screen the query must be wrapped in the matching **passthrough** envelope (`ESC P tmux; … ESC \` or `ESC P … ESC \`), or the multiplexer eats it before the underlying terminal sees it.

6.2 First implementation: /bin/sh + stty + dd

The first cut spawned a small shell helper via `uiop:run-program` that did the raw-mode dance with POSIX utilities:

```
exec 3</dev/tty 2>/dev/null || exit 1
exec 4>/dev/tty 2>/dev/null || exit 1
old=$(stty -g <&3) || exit 1
trap 'stty "$old" <&3' EXIT
stty -icanon -echo min 0 time 2 <&3
printf '\033]11;?\033\\' >&4
dd bs=64 count=1 <&3 2>/dev/null
```

Cross-platform-portable on the surface (POSIX shell, POSIX utilities), and the `stty time 2` guarantees `dd` returns in 200 ms whether the terminal answers or not. The shell handles the per-platform termios bindings that SBCL and CCL don't agree on.

6.3 The regression: SIGTTOU on macOS arm64

On macOS arm64 the helper hung at CLI startup, every time, even on a brand-new terminal session. The Ctrl-C backtrace landed in `WAIT-PROCESS`:

```
17: (PROCESS-WAIT #<SB-IMPL::PROCESS 48546 :STOPPED> NIL)
18: (UIOP/LAUNCH-PROGRAM:WAIT-PROCESS …)
19: …
22: (PROBE-TERMINAL-BACKGROUND-VIA-OSC11 :TIMEOUT-MILLIS 150)
23: (TERMINAL-BACKGROUND-LUMINANCE :PROBE-OSC11-P T)
24: (RESOLVE-COLOR-POLICY …)
25: (CLAUTOLISP.TOOLS.CLAUTOLISP:MAIN "clautolisp-sbcl" "--clautolisp")
```

The child process is `:STOPPED` — not `:EXITED`, not `:RUNNING`. That state is what the kernel sets when a process receives SIGSTOP / SIGTSTP / SIGTTIN / SIGTTOU. In our case it's **SIGTTOU**: the helper shell, running in a process group that is not the foreground group of the controlling tty, tries to modify terminal modes via `stty -icanon`. The kernel sends SIGTTOU; the shell suspends; the parent's `wait()` blocks indefinitely because the child has not exited.

Worse, `stty time` never fires because the script never reaches `dd` — it dies one step earlier. And `SIGKILL` from a hypothetical parent-side timeout would not run the `trap 'stty "$old" <&3' EXIT` cleanup, so a forced kill would leave the user's terminal in raw + no-echo mode.

6.4 The fix: opt-in via $CLAUTOLISP_PROBEOSC11

Two safe options were considered:

6.4.1 Option A: harden the helper

Wrap the subprocess in a wall-clock timeout, force-terminate on expiry, restore termios from the parent. This requires:

- A non-blocking wait loop (no portable `uiop:wait-process :timeout`).
- `uiop:terminate-process` with both `SIGTERM` and `SIGKILL` fallbacks.
- Parent-side `tcsetattr` to restore the terminal even when the child was killed before its `trap` ran. Needs per-Lisp FFI bindings.
- Detection of the SIGTTOU case specifically (vs a slow but eventually-responding terminal) so we don't burn the timeout on every invocation.

This is real engineering work and still has corner cases (what if the user backgrounds clautolisp with `Ctrl-Z` mid-probe?).

6.4.2 Option B: don't run the probe by default

The cheap rungs of the cascade — `$CLAUTOLISP_BACKGROUND` and `$COLORFGBG` — still cover the most common case (terminals that announce their background via env). When nothing else resolves, default to `:YELLOW` (dark) and let users override explicitly via the per-axis env vars. The OSC 11 helper stays in the codebase, gated behind `$CLAUTOLISP_PROBE_OSC11` so a user who knows their setup is safe can opt in.

6.4.3 What we shipped

Option B. `RESOLVE-COLOR-POLICY`'s `:probe-osc11-p` keyword defaults to `(%probe-osc11-opt-in-p)` which checks the env var. A regression test (`resolve-color-policy-does-not-spawn-osc11-probe-by-default`) monkey-patches the probe function to raise if invoked; a future caller that accidentally re-enables it surfaces as a clean test failure rather than a CI-suite hang.

The cost is **colour quality** on hosts where neither the user nor the terminal vendor sets `$COLORFGBG` or `$CLAUTOLISP_BACKGROUND`: we default to yellow-on-assumed-dark even on a white-background terminal. The win is **zero hangs**. Users who care about getting the dark/light right have two ergonomic exits — set the explicit colour vars, or set the background hint:

```
# Explicit, deterministic, no probing.
export CLAUTOLISP_SYMBOL_FOREGROUND=blue
export CLAUTOLISP_SYMBOL_BACKGROUND=default

# Or: hint the probe without enabling OSC 11.
export CLAUTOLISP_BACKGROUND=light
```

7 The per-axis explicit colour vars

`$CLAUTOLISP_SYMBOL_FOREGROUND` and `$CLAUTOLISP_SYMBOL_BACKGROUND` exist for two reasons:

1. They let users pick a specific look (e.g. bright cyan on blue) that the dark/light luminance binary cannot express.
2. They give users a deterministic exit from the probe entirely — on hosts where neither the cheap rungs nor the OSC 11 probe work (Windows console without `ENABLE_VIRTUAL_TERMINAL_PROCESSING` on the dark side of the cascade, exotic terminal multiplexers on the light side), explicit env vars are the only escape.

The vocabulary is the 16-colour palette every terminal we target supports:

- normal: `black`, `red`, `green`, `yellow`, `blue`, `magenta`, `cyan`, `white`
- bright: `bright-black` (aliased to `grey` / `gray`), `bright-red`, ..., `bright-white`

Case- and whitespace-insensitive. The sentinels `default` / `none` / `off` mean "leave this axis bare" so a user who only wants to set a background doesn't need to also spell out a foreground.

8 What about cl-charms?

We will pull in cl-charms (the Common Lisp ncurses binding) for the interactive debugger UI in a later phase. The natural question: should the OSC 11 probe also go through cl-charms / ncurses?

8.1 What ncurses does not expose

ncurses is deliberately above the terminal-control-string layer. It owns the **screen** abstraction (windows, panels, colour pairs, character cells) and has no API for sending OSC sequences or reading their responses. `tigetstr` / `tparm` / `tputs` / `setupterm` / `putp` exist for emitting terminfo capability strings but are **not currently bound** by cl-charms (marked `;; TODO` in `src/low-level/curses-bindings.lisp` around line 1721). Direct CFFI to `libncurses` would be needed for those.

ncurses' colour-pair API — `init_pair`, `start_color`, `use_default_colors` — paints the curses screen; it neither queries the terminal background nor exposes a dark/light flag. `getbkgd` returns the **curses window's** background chtype, not the terminal's RGB.

8.2 What cl-charms does expose (useful for an in-process probe)

cl-charms binds enough ncurses raw-mode primitives that we could roll an OSC 11 probe entirely in the Lisp image, no subprocess:

- `charms/ll:cbreak` / `nocbreak` / `raw` / `noraw` / `echo` / `noecho`
- `charms/ll:halfdelay` / `nodelay` / `timeout` / `wtimeout`
- `charms/ll:wgetch` / `getch` (single-char read)
- `charms/ll:def_prog_mode` / `reset_shell_mode` / `savetty` / `resetty` (state save/restore)
- High-level wrappers: `charms:enable-raw-input` (with `:interpret-control-characters`), `charms:get-char`, `charms:enable-non-blocking-mode`

To write the actual query string we'd bypass ncurses output (just `WRITE-SEQUENCE` to `*terminal-io*` followed by `FINISH-OUTPUT`); curses' `addstr~/~refresh` would mangle the escape.

8.3 Why this fixes the SIGTTOU hang

Our hang isn't `tcsetattr` per se; it's that we **forked /bin/sh** and the child's process group ended up backgrounded relative to the controlling tty. Doing the probe in-process — whether via cl-charms or via CFFI directly into libc — keeps everything inside the foreground process group that already owns the tty. ncurses' `initscr` / `newterm` do their own `tcsetattr`

from the calling process, and interactive curses apps run fine for exactly this reason. The constraint is the same: don't run the probe if your Lisp process itself has been backgrounded. In normal interactive use (which is the only context where the bg colour even matters), this is fine.

8.4 The cl-charms timing issue

OSC 11 must be probed **before** `initscr`. Once curses owns the input fd, the OSC 11 reply comes back as a stream of pseudo keypresses that `getch` surfaces as garbage. Two consequences:

1. The colour probe runs at CLI startup (it already does).
2. When the debugger later calls `initscr`, the colour decision is already cached in `*COLOR-OUTPUT*` — no re-probe.

The debugger phase will use cl-charms' raw-mode wrappers (`cbreak`, `noecho`, `wtimeout`, `getch`) standalone **outside** curses initialisation to do the OSC 11 round trip. They're thin termios wrappers; nothing requires us to be in curses mode to call them.

8.5 Migration plan

When cl-charms lands as a runtime dependency for the debugger:

- Add `PROBE-TERMINAL-BACKGROUND-VIA-CHARMS` alongside the shell-based helper.
- Default `%PROBE-OSC11-OPT-IN-P` to `T` when the new helper is available **and** the host is Unix (see the Windows section below).
- Keep the shell-based helper as a fallback for builds without cl-charms.
- Update the regression test to verify the new helper isn't called from non-tty contexts.

9 MS-Windows portability

Three sub-targets to consider, each with its own colour story:

9.1 Pure Windows (cmd.exe / PowerShell / Windows Terminal, native build)

- No `/bin/sh`. The current shell helper errors out of `uiop:run-program`, `handler-case` catches it, the cascade falls through to `:YELLOW`. Benign — no hang.
- The real concern is **colour output itself**: ANSI SGR sequences only render on Windows if `ENABLE_VIRTUAL_TERMINAL_PROCESSING` is set on the console handle. Windows Terminal turns it on by default; legacy conhost on Win10 1809+ does too; older builds don't. On those we would emit literal `^[[33m` text. So the right Windows gate is "is VT processing enabled" **before** we even consider OSC 11.
- Windows Terminal does in fact answer OSC 11 (Microsoft added it around 2020), so the **concept** is portable; only the **transport** needs replacing: Win32 console API (`GetConsoleMode`, `SetConsoleMode`, `WriteConsoleW`, `ReadConsoleInputW`) rather than `tcsetattr` + `/dev/tty`.

9.2 msys2 / Git Bash via mintty

- Mintty / msys runs without job control, so SIGTTOU isn't sent — the shell helper that hung on macOS may **just work** here. Mintty does respond to OSC 11. A user who sets `CLAUTOLISP_PROBE_OSC11=1` on Git Bash will likely get a clean answer.
- But `/dev/tty` handling in msys is non-trivial (named-pipe shim, not a real device file in some cases), and the lack of unix permissions / job control means failure modes are different from Linux. The right long-term answer is the same Win32 path as case 1 — msys is running in a real Windows console regardless of the user-space veneer.

9.3 WSL

Full Linux — the shell helper works the same as on Linux. The OSC 11 opt-in stays on the user.

9.4 cl-charms on Windows

cl-charms wraps ncurses via CFFI. Windows installations of ncurses are either:

- **PDCurses** — a re-implementation that knows nothing about OSC or ANSI; it draws directly to the Win32 console. Useful for the **debugger UI** on Windows, useless for OSC 11.
- **mingw-ncurses** — works only inside msys2.

Neither helps with the pure-Windows case, which is where most Windows users will land. When the debugger ships we'll likely have:

- A cl-charms-via-ncurses path for Unix (OSC 11 probe via cl-charms raw-mode wrappers).
- A separate Win32-console path for Windows (OSC 11 probe via `Read~/~WriteConsole*` once VT processing is on; PDCurses for the debugger UI on top).

Both paths converge on the same `*COLOR-OUTPUT*` semantics.

9.5 Immediate Windows work (not done yet)

The current state on Windows is "no probe, default `:YELLOW`, may emit literal escapes in pre-VT consoles." That's wrong but not broken. The cleanest fix is to add a Win32 branch to `OUTPUT-STREAM-IS-TTY-P` / `RESOLVE-COLOR-POLICY` that calls `GetConsoleMode` and only returns "tty-ish" when `ENABLE_VIRTUAL_TERMINAL_PROCESSING` is on (or toggles it on first). This rides with the cl-charms / debugger work rather than being its own ticket.

10 What we did not do (and why)

10.1 Active probing by default

See "Why the OSC 11 probe is opt-in" above. Sums to: the cost of a wrong default (CLI hang on macOS) dwarfs the benefit of a right default (auto-pick yellow vs blue on the rare host without `$COLORFGBG`).

10.2 256-colour or truecolour palette

The 16-colour palette covers every terminal we target without capability negotiation. Truecolour would need a `$COLORTERM` check and an SGR `38;2;R;G;B` emission path — worthwhile if a user asks, not worth shipping speculatively.

10.3 Bold / italic / underline attributes

Not yet requested. The SGR helpers are written so that extending `%ANSI-SGR-PARTS` to handle attribute keywords (`:bold`, `:underline`, …) is a one-table change.

10.4 Colouring elements other than symbols

The current scope is **AutoLISP symbols only**. Strings, numbers, `SUBR` wrappers, etc. are uncoloured by deliberate choice: backtraces like `in SUBR CAR: (A-SYMBOL)` already compose through Common Lisp's list printer, so colouring only the symbol gives us visual structure without painting the whole backtrace. Future elements would get their own `$CLAUTOLISP_<ELEMENT>_FOREGROUND` env vars and their own `*<element>-color*` parameters.

10.5 Detecting tmux / screen for the output sequence

The OSC 11 **query** needs a passthrough wrapper inside tmux / screen because we want a reply. The **output** SGR sequences do not — tmux passes them through to the underlying terminal unchanged. So our colour emission is multiplexer-blind by design.

11 Test surface

Located in `clautolisp/autolisp-runtime/tests/model-tests.lisp`:

- `ansi-colorize-no-ops-when-colour-is-nil` — the cheap path for non-coloured runs.
- `ansi-colorize-wraps-string-in-sgr-sequence` — single- keyword foreground form.
- `ansi-colorize-renders-foreground-and-background-pair` — `(FG BG)` list form, bright variants, bg-only, unknown colours.
- `print-object-symbol-uses-colour-when-policy-armed` — the integration with `AUTOLISP-SYMBOL`'s `PRINT-OBJECT`.
- `print-object-symbol-renders-with-foreground-and-background` — same, with both axes.
- `parse-colour-name-handles-case-whitespace-sentinels` — name parser tolerances.

- `parse-colorfgbg-classifies-background-index` — rxvt env parser.
- `parse-osc11-response-extracts-rgb-triple` — both ESC-ST and BEL response terminators.
- `luminance-of-rgb-matches-rec-601` — luminance maths.
- `env-symbol-colour-spec-pairs-the-two-env-vars` — the new per-axis env vars.
- `resolve-color-policy-honours-no-color-flag` — CLI flag short-circuit.
- `resolve-color-policy-honours-no-color-env` — `$NO_COLOR` short-circuit.
- `resolve-color-policy-honours-symbol-colour-env-vars` — explicit colours win over the luminance probe.
- `resolve-color-policy-does-not-spawn-osc11-probe-by-default` — regression for the SIGTTOU hang; monkey-patches the probe function to raise on invocation.

The OSC 11 probe itself is not unit-tested because doing so requires a controlling tty inside the test runner, and any tty flake there is exactly the kind of fragility we want to avoid. The probe is exercised manually by setting `$CLAUTOLISP_PROBE_OSC11=1` in an interactive session.

12 See also

- `clautolisp/autolisp-runtime/source/terminal-color.lisp` — the implementation.
- `clautolisp/documentation/clautolisp-specifications.org` — the user-facing spec for the same machinery.
- `clautolisp/documentation/clautolisp-user-manual.org` — the menu of env vars with worked examples.
- `autolisp-front-end/documentation/alfe--specifications.org` — the alfe-side env-var table.
- `issues/closed/debug-messages-autolisp-symbols.issue` — the original bug, the colour follow-up, and the SIGTTOU hotfix.

- `https://no-color.org` — the universal no-color convention.
- `xterm(1)` — OSC 11 specification.
- `tmux(1)` — DCS passthrough envelope used to escape OSC sequences out of the multiplexer.
- `https://github.com/HiTECNOLOGYs/cl-charms` — cl-charms source.
- curs$_{\text{initscr}}$(3x) — what ncurses' `initscr` does to termios.
- nelhage on termios + job control — explains SIGTTOU.